

Improvement and Revision Guide

What to improve, what to revise, and how to impress the interviewer

Project	Production-Ready Containerized Inventory & Order Management System
Stack	React, FastAPI, PostgreSQL, Docker, Docker Compose, Git
Purpose	First-round software engineer technical assessment and interview preparation

How to use this document

Use this document as a practical checklist before submission and as talking points during the technical interview. It focuses on what is complete, what still needs attention, and what can be added after submission without creating unnecessary risk.

Improvement Plan That Attracts Interviewers

Interviewers usually like projects that show correct fundamentals, production thinking, and awareness of future improvements. The aim is not to add everything immediately; the aim is to explain priorities clearly.

Improvement	What to add	Impact	Effort	Why interviewer likes it
README Screenshots	Add screenshots of login, dashboard, products, customers, orders, order details modal, Swagger docs, Docker Compose running.	High	Low	Makes project easier to review quickly.
Demo Data Seed	Provide seed.py and README command for demo admin, products, customers, and sample order.	High	Low	Reviewer can test immediately.
Live Health Links	Add /health and /health/db URLs in README.	High	Low	Shows deployment is working.
CI/CD Tests	GitHub Actions to run pytest and npm build on push.	High	Medium	Shows professional workflow.
Alembic Migrations	Replace create_all-only workflow with proper migrations.	High	Medium	Strong production-readiness topic.
RBAC	Admin/viewer permissions for sensitive actions.	High	Medium	Improves security design.
Audit Log	Track every create/update/delete/status change with user and timestamp.	High	Medium	Very attractive enterprise feature.
Stock Ledger	Separate stock_movements table for every inventory change.	High	Medium	Makes inventory tracking robust.
Reports Export	CSV/PDF export for orders, products, customers, low stock.	Medium	Low	Useful business feature.
Date Filters	Filter orders by date range, month, customer, status.	Medium	Low	Improves data usability.
Refresh Token Flow	Short-lived access token + refresh token.	Medium	High	Advanced security feature.
Monitoring	Structured logs, request ID, error tracking.	Medium	Medium	Production operations topic.

Best Improvements to Mention in Interview

- I would add an inventory ledger table so every stock movement is traceable. This is better than only storing the current product quantity.
- I would add role-based access control so admin users can modify data and viewer users can only read data.
- I would move schema changes fully to Alembic migrations so deployment does not depend on automatic create_all.
- I would add CI/CD using GitHub Actions to run tests and frontend build before merge.
- I would add audit logs for compliance and debugging, especially for stock and order status changes.

Technical Topics to Revise Before Interview

Topic	Revise these points	Interview focus
FastAPI	Routers, dependencies, response_model, status codes, exception handling, middleware, OpenAPI docs.	Be able to explain why FastAPI fits REST APIs.
Pydantic	BaseModel, Field validation, EmailStr, validators, ConfigDict, schema normalization.	Explain request validation and data normalization.
SQLAlchemy	ORM models, relationships, sessions, commit/rollback, IntegrityError, eager loading, with_for_update.	Very important for backend interview.
PostgreSQL	Primary key, foreign key, unique constraints, check constraints, indexes, transactions, isolation.	Explain data integrity and concurrency.
Inventory logic	Stock deduction, insufficient stock, stock restore, order cancellation, backend total calculation.	Core business logic of your project.
JWT Auth	Password hashing, token creation, token expiry, Bearer token, protected routes, secret key.	Security-related questions are common.
HTTP/REST	GET/POST/PUT/PATCH/DELETE, 200/201/204/400/401/403/404/409/422.	Basic API interview area.
Docker	Dockerfile layers, slim images, Compose services, networks, volumes, environment variables.	Mandatory assessment requirement.
Deployment	Render backend, Vercel frontend, Docker Hub image, CORS, VITE_API_URL, DATABASE_URL.	You must explain how live app is hosted.
React	useState, useEffect, useMemo, props, component structure, forms, conditional rendering, modals.	Frontend discussion expected.
Axios/API handling	Base URL, token interceptor, error handling, 401 redirect, timeout.	Good practical frontend topic.
Testing	pytest, TestClient, fixtures, dependency overrides, test DB, business logic tests.	Proves quality.

Seven-Day Revision Plan

Day	Theme	Work to do
Day 1	Project walkthrough	Practice explaining architecture, modules, and demo flow in 2 minutes.
Day 2	Backend and business logic	Revise product/customer/order APIs, stock deduction, insufficient stock, and total calculation.
Day 3	Database	Revise SQLAlchemy models, relationships, constraints, transactions, and indexes.
Day 4	Authentication and security	Revise JWT, password hashing, CORS, environment secrets, and protected routes.
Day 5	Docker and deployment	Revise Dockerfile, Docker Compose, db hostname vs localhost, Render, Vercel, Docker Hub.
Day 6	React frontend	Revise hooks, forms, state updates, Axios wrapper, pagination, dashboard, and modal.
Day 7	Mock interview	Practice 30 questions, run live demo, explain improvements and limitations honestly.

High-Impact Talking Points

- I kept business rules on the backend because frontend validation alone is not secure.
- I used unique constraints in the database and also returned user-friendly 409 conflict errors.
- I used Docker Compose to run frontend, backend, and PostgreSQL together, making setup reproducible.
- I added tests for the most important business rules: duplicate SKU/email, insufficient stock, stock reduction, and stock restoration.
- I used environment variables for deployment configuration to avoid hardcoding credentials.
- I would add Alembic, RBAC, audit logs, and a stock ledger as next production improvements.

Topics You Should Be Ready to Draw on Paper

- Architecture diagram: Browser -> React -> FastAPI -> SQLAlchemy -> PostgreSQL.
- Database ER diagram: users, products, customers, orders, order_items.
- Order creation flow: validate customer -> validate products -> lock stock -> calculate total -> reduce quantity -> create order items -> commit.
- Docker Compose network: frontend, backend, db service, named volume, environment variables.
- JWT flow: login -> token -> Authorization header -> get_current_user dependency -> protected route.

Final Submission Readiness Checklist

Checklist item	Expected result	Notes
Backend tests pass	30 passed	Before deployment and before final commit.
Frontend build passes	npm run build	Before Vercel deployment.
Docker Compose works	docker compose up --build	Before pushing final README.
Docker Hub image pushed	docker push username/inventory-backend	Required deliverable.
Render backend live	/health and /docs work	Required deliverable.
Vercel frontend live	Login and API calls work	Required deliverable.
README updated	Add URLs, demo login, screenshots, commands	Professional presentation.
Assessment form submitted	GitHub, Docker Hub, frontend URL, backend URL	Final step.